



Revision Notes: Fundamentals of Recursion in Software Engineering

Introduction to Recursion

Recursion is a fundamental concept in computer science and software engineering. It refers to the process where a function calls itself in order to solve a problem. Recursion is used when a problem can be broken down into similar subproblems. If these subproblems are solved, the larger problem can also be solved. This technique is particularly useful when dealing with problems of repetitive nature and allows for elegant solutions to complex issues

【4:2+transcript.txt】.

Example of Recursion

One of the classic examples of recursion is the calculation of the factorial of a number. The factorial of n , denoted as $n!$, is the product of all positive integers less than or equal to n . This can be recursively defined as:

- $\text{fact}(n) = n * \text{fact}(n-1)$
- Base Condition: $\text{fact}(0) = 1$

The base condition is crucial as it stops the function from calling itself indefinitely. It defines the simplest instance of the problem which can be solved directly without any further recursion

【4:8+transcript.txt】 【4:13+transcript.txt】.

Function Call Tracing

In recursive functions, it's important to understand how each function call is managed. This is where stack data structures come into play. Each recursive call to a function creates a new stack frame. These frames scaffold on top of each other in the system's



For example, when calculating `factorial(3)` , the call stack grows as follows:

1. `fact(3)` calls `fact(2)`
2. `fact(2)` calls `fact(1)`
3. `fact(1)` calls `fact(0)` , which returns 1 (base case)
4. Then the stack unwinds, calculating the results:
 - `fact(1)` gets 1 from `fact(0)` , computes $1 * 1$, and passes it back.
 - `fact(2)` gets 1 from `fact(1)` , computes $2 * 1 = 2$, and passes it back.
 - `fact(3)` gets 2 from `fact(2)` , computes $3 * 2 = 6$, and completes **【4:1+transcript.txt】 【4:14+transcript.txt】** .

Applying Recursion to Solve Problems

Printing Numbers Recursively

Recursion can be used for tasks such as printing a series of numbers. For instance, to print numbers from 1 to n:

```
void printNumbers(int n) {  
    if (n > 0) {  
        printNumbers(n - 1);  
        System.out.println(n);  
    }  
}
```

This code prints numbers in ascending order by recursively calling `printNumbers(n - 1)` until `n` is 0, then unwinds, printing `n` at each step **【4:12+transcript.txt】** .

Complexity in Recursion

Time Complexity

The time complexity of recursion can be challenging to determine directly due to the repeated nature and multiple recursive calls. It



complexity is often represented as $O(n)$ or exponential, depending on the number of recursive calls and operations performed per call 【4:16+transcript.txt】 【4:5+transcript.txt】 .

Space Complexity

Space complexity in recursion is driven by the maximum depth of the stack. The deeper the recursion, the more space it requires due to the stack frames. For example, calculating Fibonacci numbers recursively has a space complexity proportional to $O(n)$ due to the deep call structure. However, this can often be optimized with techniques such as memoization 【4:11+transcript.txt】 【4:15+transcript.txt】 .

Key Takeaways

- **Base Case:** Essential to terminate recursion and avoid infinite loops.
- **Stack Frames:** Each recursive call generates a new frame, utilizing memory.
- **Recursion vs Iteration:** Recursive solutions are often more elegant but may be less efficient due to overhead of multiple function calls and stack size limitations.
- **Practical Use:** Widely used in algorithms, particularly in divide-and-conquer techniques like file system traversal, backtracking, etc.

By mastering recursion, software engineers can develop solutions that are efficient and scalable to solve complex, repetitive problems in computation.